NFL Scheduler

Background

The NFL schedule determines when each team plays its regular-season games. Before scheduling begins, the league already knows each team's opponents and whether each matchup will be played at home or away. These matchups are determined after the previous season ends using the NFL's official formula, which accounts for division membership, rotating conference matchups, and prior-year standings. As a result, the scheduler's job is not to choose opponents, but to assign each predetermined game to a specific week and time slot.

For this project, we collected the full set of NFL matchups for the season and paired them with the available broadcast time slots. Because each of the 32 teams plays 17 games across an 18-week season, with one bye week, the scheduler must place every game while satisfying a wide range of practical and competitive constraints. We treated predicting the number of Sunday 1:00 PM versus 4:00 PM windows as out of scope, since that would add substantial complexity without changing the core scheduling problem.

After assembling the data, we researched and encoded several important scheduling rules. These include ensuring that each team plays at most once per week, each game occurs exactly once, the Super Bowl champion opens at home on Thursday night, teams do not have four straight road games, teams sharing a stadium are not both home at the same time, teams do not play three games in eleven days, West Coast and Mountain Time home teams do not host early Sunday games, the Lions and Cowboys host on Thanksgiving in the appropriate order, Week 18 consists entirely of divisional matchups, bye weeks fall between Weeks 4 and 14, and teams avoid excessive cross-country travel across consecutive weeks.

Initial approach

Our first implementation used a SAT formulation. We modeled every possible game-slot assignment as a Boolean variable, where a variable is true if a particular game is assigned to a particular slot. With 272 games and 126 slots, this produced a large but manageable variable space. We then built the model constraint by constraint using helper methods such as exactuly_one, at_most_one, at_least_one, and at_most_k. . For example, "each game happens exactly once" was enforced by requiring exactly one of the slot variables for each game to be true, while slot capacities were enforced with at_most_k constraints. Team weekly participation limits, the Super Bowl opener, no four straight road games, stadium conflicts, short-week limits, Week 18 divisional restrictions, and time-zone kickoff rules were all encoded similarly through combinations of these helper constraints. More complex rules, such as avoiding three games in

eleven days and preventing excessive coast-to-coast travel swings, required sliding-window constraints across multiple weeks and were much more difficult for the solver.

We also began developing a CP-SAT style solver to see whether a constraint programming approach would be more effective, especially because it could also support the second phase of the project: maximizing TV viewership. In that version, games and slots were modeled as integer assignments rather than Boolean variables, using AddInverse constraint to map games to slots and slots to games. In practice, however, this formulation was substantially slower in its current state, with only a small subset of constraints solving within the same time limit.

To support the complex scheduling rules, we introduced a number of helper Boolean variables derived from integer assignments. For example, we constructed indicator variables for whether a game occurs in a given week, day, or time zone using *AddAllowedAssignments* and related constraints. These were then aggregated into team-level helper variables such as whether a team plays in a given week, plays on a specific day, or plays a road game. This structure allowed us to express higher-level constraints (such as limiting teams to one game per week, preventing four consecutive road games, and restricting short-week occurrences) using sums and logical relationships over these helper variables.

More complex temporal constraints, such as preventing three games within eleven days or limiting cross-country travel oscillations, required chaining together multiple week-based indicators. For example, the “no three games in eleven days” rule was encoded using disjunctive constraints across Monday/Sunday/Thursday patterns spanning three weeks. Similarly, travel constraints were modeled by tracking time zone transitions week-to-week and disallowing sequences with excessive cumulative distance. While expressive, these constraints significantly increased the model’s complexity and search space.

One advantage of the CP-SAT formulation is its ability to naturally incorporate optimization. We extended the model to maximize projected TV viewership by assigning each game a quality score (based on team market size and rivalries) and each slot a value (based on time and prominence). Using *AddElement*, we mapped each game’s assigned slot to a corresponding score and maximized the total across all games. This objective integrates seamlessly with the constraint system, allowing the solver to search for high-quality feasible schedules.

Challenges

The SAT model handled simpler constraints well, but performance degraded once we introduced constraints with dependencies across multiple weeks. In cumulative runs, the early constraint blocks—game uniqueness, slot capacity, one game per week, Thanksgiving placement, the Super Bowl opener, no four straight road games, and stadium conflicts—solved quickly. The first major bottleneck appeared with the “three games in eleven days” constraint, after which later cumulative models often timed out under a 15-second limit.

The most difficult constraints were those involving multi-week windows, especially the three-games-in-eleven-days rule and the cross-country “ping-ponging” rule. To improve performance, we rewrote some of these constraints using cached helper variables, replaced long clauses with derived “or” variables, and relaxed the ping-ponging rule so that it forbade only a smaller set of especially undesirable travel patterns. These changes made the model more tractable, but solving remained slow overall.

Even so, the optimized model represented a meaningful improvement: instead of failing to find a solution after hours, it produced a valid schedule in about 15 minutes. As a result, we narrowed the scope of the project. Rather than generating many random schedules satisfying our SAT constraints, we focused on producing a single feasible schedule.

Results:

The solver outputs are evaluated by comparing each generated schedule against the official NFL schedule across several levels of strictness. The metrics include exact matchup placement by week, exact matchup placement by week/day/time, matching the same kickoff slot regardless of week, matching broad time windows such as lunch, afternoon, or night, and matching whether each home team appears in the correct week or exact slot. The evaluation also checks team-level structure through bye-week accuracy and consecutive-game-pair accuracy, where each team’s 18-week sequence, including its bye, is broken into adjacent pairs and compared to the official schedule regardless of where those pairs occur. Each metric is reported as a percentage, with an expected random-schedule baseline shown alongside it to indicate whether the solver is doing better than chance under the same general slot inventory.

Metric	**SAT Results**	**CP Results**	**Random Baseline**
Correct week overall	20/272 (7.35%)	8/272 (2.94%)	~5.58%
Correct week and time slot	8/272 (2.94%)	3/272 (1.10%)	~1.50%
Correct time slot (any week)	94/272 (34.56%)	88/272 (32.35%)	~25.65%
Correct time window (any week/day)	124/236 (52.54%)	112/236 (47.46%)	~36.95%

Correct home team (correct week)	121/272 (44.49%)	121/272 (44.49%)	~39.14%
Correct home team (exact slot)	44/272 (16.18%)	35/272 (12.87%)	~11.79%
Correct bye week by team	1/32 (3.12%)	3/32 (9.38%)	~10.00%
Correct consecutive game pairs	42/544 (7.72%)	42/544 (7.72%)	~5.56%

SAT is generally better than a random baseline, but still far from reaching 100% that would represent the actual NFL schedule. This shows that our constraints do push us towards the NFL schedule, but there is clearly a broad search space even with all of these constraints that SAT struggled with.

The CP-SAT solver with TV viewership optimization performs similarly to the SAT solver in terms of schedule accuracy, with slightly lower performance on most metrics compared to SAT. The CP solver achieved 8/272 (2.94%) correct week placements and 3/272 (1.10%) correct week/day/time slot placements, compared to SAT's 20/272 (7.35%) and 8/272 (2.94%) respectively.

Interestingly, CP performs better on bye week accuracy (3/32 teams = 9.38% vs SAT's 3.12%) and matches SAT exactly on consecutive game pair accuracy (42/544 = 7.72%). The CP solver successfully finds feasible schedules that respect all NFL constraints while optimizing for TV viewership, placing high-market teams and rivalries in prime-time slots. The viewership analysis shows the top prime-time matchups include high-value rivalries like NYG @ DAL (Week 4 Monday) and DAL @ PHI (Week 16 Monday), demonstrating that the optimization objective successfully prioritizes valuable matchups for premium time slots.

Solver

The solver took a long time to find a feasible schedule, especially under certain constraint formulations, such as the original version of the cross-country “ping-ponging” rule. Our initial SAT formulation did not solve within several hours. After reformulating a few of the more complex constraints, however, the model was able to produce a solution in about 10–15 minutes.

The resulting CNF formula was still large, with 216,610 variables and 725,607 clauses encoding the full set of constraints. Although the model contains many restrictions, the generated schedules were still quite different from the actual NFL schedule, suggesting that the feasible search space remains large and diverse. Therefore, a solver more specifically optimized for this scheduling problem may be able to search that space more efficiently and find feasible, or even higher-quality, solutions much more quickly.

The CP-SAT solver with viewership optimization completes in approximately 5 minutes with an 8-worker parallel search configuration, significantly faster than the SAT solver's 10-15 minute runtime. The CP model uses OR-Tools' constraint programming framework, which provides more expressive modeling capabilities for complex scheduling constraints compared to the Boolean SAT formulation.

The CP approach allows for direct modeling of complex NFL rules using integer variables and constraint types like *AddForbiddenAssignments*, *AddMaxEquality*, and *AddBoolOr*, making the code more readable and maintainable than the SAT's CNF clause generation. The viewership optimization is implemented as a maximization objective that combines team market values, rivalry bonuses, and slot importance scores, successfully placing 56 games in prime-time slots with an average market value of 60 and 12 rivalry matchups.

While CP-SAT finds feasible solutions more quickly than SAT, the resulting schedules remain quite different from the official NFL schedule, suggesting that additional domain-specific heuristics or constraint tightening would be needed to achieve higher accuracy.

Code

1. SAT

nfl_sat.py builds an NFL schedule by encoding the problem as a Boolean SAT formula. Each main variable represents whether a specific game is assigned to a specific time slot, and the code adds CNF clauses to enforce hard scheduling rules: every game is scheduled once, slot capacities are respected, teams play at most once per week, no invalid bye weeks occur, stadium-sharing teams do not host on the same date, Thanksgiving and opening-game requirements are met, week 18 contains only divisional games, and travel/short-week constraints are limited. Helper variables are used to represent higher-level conditions like “team plays in this week” or “team has a short week,” and cached indices make those constraints easier to build. After constructing the formula, the script runs a PySAT solver and decodes the satisfying assignment back into a readable week-by-week schedule.

2. CP

nfl_cp_sat.py implements an NFL schedule optimizer using OR-Tools' CP-SAT constraint programming solver, extending the basic scheduling constraints with TV viewership

optimization. The class NFLSchedulerCPSAT builds a constraint model that assigns 272 games to 272 time slots while respecting all NFL scheduling rules and maximizing viewership potential. We load game matchups and time slots from CSV files and precompute indices for efficient constraint building. We use integer variables for game-to-slot assignments and boolean variables for complex team-week relationships. Our constraints include the 11 NFL scheduling rules including team availability, travel restrictions, bye weeks, stadium conflicts, and special event requirements (Thanksgiving, Week 18 divisions). We added market value scoring (32 teams rated 30-100), rivalry detection (30 key matchups with 20-point bonuses), and slot importance hierarchy (Thursday Night Football=100 to international games=10). Our objective function maximizes total viewership score by placing high-value matchups in prime-time slots.

3. Accuracy

accuracy.py evaluates a generated schedule by comparing it against the official NFL schedule using several accuracy metrics. It loads both schedules into a common game format, precomputes matchup maps and counters, then measures how often the generated schedule matches the official one by matchup week, exact week/day/time slot, same time slot regardless of week, broad time window, home team in the correct week or exact slot, bye week, and consecutive team schedule pairs. Each metric also reports an expected random baseline using the same slot inventory, which helps show whether the solver is doing better than chance rather than only reporting raw percentages.

Hypothesis

We theorized that if we optimized for TV viewership by prioritizing high-market teams and rivalries in prime-time slots, we could generate schedules that better align with the business logic driving actual NFL scheduling decisions. The viewership optimization worked because we analyzed the output and confirmed that high-value rivalries genuinely appeared in premium slots more often than low-value matchups. This suggests our optimization objective was sound. However, maximizing TV viewership alone is insufficient to replicate the official NFL schedule, indicating that the league's scheduling logic incorporates factors beyond just market values and rivalries.

File Structure

The csv folder holds the raw data our solver uses, specifically a csv containing all of the time slots, a csv containing all of the regular season games, and a csv containing the actual schedule. Additionally, our outputs for both sat and CP are in the outputs folder. The other files in our final project are at the root and contain the code for sat (nfl_sat.py), cp (nfl_cp_sat.py, run_solver.py, analyze_veiwership.py, and test_cp_sat.py), as well as the accuracy (accuracy.py).

Instructions to run final project:

SAT: python nfl_sat.py (can take a very long time)

Accuracy on SAT results: python accuracy.py

Accuracy on CP SAT results: python accuracy.py outputs/solved_schedule_cp_viewership.txt

CP-SAT with Viewership Optimization: python run_solver.py (runs in ~5 minutes, outputs to solved_schedule_cp_viewership.txt)

python analyze_viewership.py (analyzes prime-time quality of the optimized schedule)

python test_cp_sat.py (validates viewership algorithms)